

ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 10/08

- Problema de Omega de 2do Parcial.
- Ejercicios de Teoría de 2do Parcial.
- Revisión de dudas.
- Arreglos multidimensionales

2

Algoritmos y Estructuras de Datos – AEDD – Calendario

Instancias de Evaluación	Parte	Fecha
Parcial 2	Parte Teórica Escrita	03/08/2026
	Parte Práctica Escrita	03/08/2026
	Parte Sitio Juez	03/08/2026
Parcial 2 - Recuperatorio	Parte Práctica Escrita	20/08/2026 14:30 hs
	Parte Sitio Juez	21/08/2026 8:00 hs
Parcial 3	<i>En el mes de septiembre (final) (fecha a confirmar)</i>	

3

Algoritmos y Estructuras de Datos – Revisión dudas parcial 2

- ✓Problema Omega Parcial 2: Diferentes soluciones.
- ✓Revisión ejercicios de teoría.
- ✓Dudas?

4

Algoritmos y Estructuras de Datos – AEDD – Asistencia 10/08

Contraseña del estudiante mi6kho

5

Problema en Omega

Revisen todos el enunciado, y me indican diferentes alternativas de solución.

- ¿Tamaño físico del arreglo? ¿Tipo de dato?
- ¿Tamaño lógico del arreglo?
- ¿Cómo recorren el arreglo?

6

Revisemos resoluciones

Dudas????

7

Repaso con preguntas de Teoría

Ejercicio 1 - Verdadero y Falso con Justificación.

Para cada uno de los siguientes enunciados indique V o F. En todos los casos, justifique su respuesta.

- A. Es posible escribir un programa en C++ sin hacer uso de ningún tipo de funciones.
- B. En C++, no es posible retornar más de un valor desde una función. La única forma de devolverle información a quien invoca es mediante la sentencia return.
- C. La operación de eliminación de un elemento en un vector que se maneja con TL siempre implica que al menos un elemento debe moverse hacia la izquierda.
- D. Si declaramos un arreglo estático de 5 elementos y en un ciclo intentamos asignar un valor a un elemento ubicado en una posición que excede el tamaño físico, el compilador de C++ no detectará ningún error en tiempo de compilación, ni tendremos errores en tiempo de ejecución, resultando en una ejecución exitosa.

8

Repaso con preguntas de Teoría

1. ¿Qué afirmación es verdadera con respecto al paso de parámetros por referencia?

- Si en la definición del parámetro formal introduzco la palabra reservada 'const', la variable no se podrá modificar dentro de la función.
- Es el pasaje de parámetro por default para variables de tipo string.
- Si el parámetro es modificado, solo se ve afectado dentro del cuerpo de la función.
- Si un parámetro formal es definido por referencia, debo obligatoriamente definir todos los parámetros siguientes de la misma manera (es decir, por referencia).

9

Repaso con preguntas de Teoría

1. ¿Qué afirmación es verdadera con respecto al paso de parámetros por referencia?

- *Si en la definición del parámetro formal introduzco la palabra reservada 'const', la variable no se podrá modificar dentro de la función.*
- Es el pasaje de parámetro por default para variables de tipo string.
- Si el parámetro es modificado, solo se ve afectado dentro del cuerpo de la función.
- Si un parámetro formal es definido por referencia, debo obligatoriamente definir todos los parámetros siguientes de la misma manera (es decir, por referencia).

10

Repaso con preguntas de Teoría

1. Cuál de las siguientes descripciones representa correctamente el comportamiento principal del método de ordenación por "Inserción Directa"?
 - Se selecciona el elemento más pequeño de todo el arreglo y se lo intercambia con el elemento de la primera posición.
 - Se consideran pares de elementos adyacentes y se intercambian si están en el orden incorrecto, llevando los valores más chicos hacia la izquierda repetidas veces.
 - Se divide el arreglo original en dos partes iguales, se ordenan ambas partes por separado y luego se intercalan.
 - Se considera un elemento a la vez y se lo desplaza hacia su lugar correcto entre los elementos anteriores, los cuales ya se mantienen ordenados.

11

Repaso con preguntas de Teoría

1. Cuál de las siguientes descripciones representa correctamente el comportamiento principal del método de ordenación por "Inserción Directa"?
 - Se selecciona el elemento más pequeño de todo el arreglo y se lo intercambia con el elemento de la primera posición.
 - Se consideran pares de elementos adyacentes y se intercambian si están en el orden incorrecto, llevando los valores más chicos hacia la izquierda repetidas veces.
 - Se divide el arreglo original en dos partes iguales, se ordenan ambas partes por separado y luego se intercalan.
 - *Se considera un elemento a la vez y se lo desplaza hacia su lugar correcto entre los elementos anteriores, los cuales ya se mantienen ordenados.*

12

Repaso con preguntas de Teoría

3. Tu compañero se encuentra en clase de práctica de AEDD implementando un algoritmo de ordenamiento burbuja para ordenar un arreglo `arr` de enteros de mayor a menor y te ha preguntado qué condición de ordenamiento debe utilizar para lograrlo. Suponiendo que ha definido los for anidados de la siguiente manera:

```
for (int i = 0; i < n - 1; i++)
```

```
    for (int j = 0; j < n - i - 1; j++)
```

¿Qué le responderías para que su solución sea aceptada en OmegaUp?

- `if (arr[j] > arr[j + 1])`
- `if (arr[j] < arr[j + 1])`
- `if (arr[i] > arr[j])`
- `if (arr[i] < arr[j + 1])`

13

Repaso con preguntas de Teoría

3. Tu compañero se encuentra en clase de práctica de AEDD implementando un algoritmo de ordenamiento burbuja para ordenar un arreglo `arr` de enteros de mayor a menor y te ha preguntado qué condición de ordenamiento debe utilizar para lograrlo. Suponiendo que ha definido los for anidados de la siguiente manera:

```
for (int i = 0; i < n - 1; i++)
```

```
    for (int j = 0; j < n - i - 1; j++)
```

¿Qué le responderías para que su solución sea aceptada en OmegaUp?

- `if (arr[j] > arr[j + 1])`
- **`if (arr[j] < arr[j + 1])`**
- `if (arr[i] > arr[j])`
- `if (arr[i] < arr[j + 1])`

14

Repaso con preguntas de Teoría

4. Seleccione la opción que completa esta afirmación de forma correcta: “Si en una función F() declaro un arreglo `int arr[10]` y luego llamo a otra función G() pasando `arr` como parámetro...”

- ... el programa genera un error, ya que no se pueden pasar arreglos entre funciones.
- ... ambas funciones pueden acceder a la misma memoria física del arreglo.
- ... no importa qué cambios aplique G() sobre el arreglo. Al finalizar la llamada el arreglo permanecerá con los datos iniciales ya que en C++ los parámetros de las funciones pasan por copia por defecto.
- ... el programa genera un error ya que no se pueden declarar arreglos dentro de funciones, únicamente es posible dentro de `main()`.

15

Repaso con preguntas de Teoría

4. Seleccione la opción que completa esta afirmación de forma correcta: “Si en una función F() declaro un arreglo `int arr[10]` y luego llamo a otra función G() pasando `arr` como parámetro...”

- ... el programa genera un error, ya que no se pueden pasar arreglos entre funciones.
- ... *ambas funciones pueden acceder a la misma memoria física del arreglo.*
- ... no importa qué cambios aplique G() sobre el arreglo. Al finalizar la llamada el arreglo permanecerá con los datos iniciales ya que en C++ los parámetros de las funciones pasan por copia por defecto.
- ... el programa genera un error ya que no se pueden declarar arreglos dentro de funciones, únicamente es posible dentro de `main()`.

16

Repaso con preguntas de Teoría

5. ¿Cuál afirmación describe correctamente una función recursiva?

- Debe llamarse a sí misma con al menos un caso base para evitar recursión infinita.
- No puede tener parámetros de tipo referencia.
- Siempre utiliza más memoria que una solución iterativa por defecto.
- El caso recursivo debe resolver de forma directa un subproblema de mayor complejidad que el caso base.

.

17

Repaso con preguntas de Teoría

5. ¿Cuál afirmación describe correctamente una función recursiva?

- ***Debe llamarse a sí misma con al menos un caso base para evitar recursión infinita.***
- No puede tener parámetros de tipo referencia.
- Siempre utiliza más memoria que una solución iterativa por defecto.
- El caso recursivo debe resolver de forma directa un subproblema de mayor complejidad que el caso base.

.

18

Repaso con preguntas de Teoría

6. Seleccione la afirmación que es falsa:

- El prototipo de una función le indica, formalmente, a la función que la invoca el tipo de dato que será devuelto si es que hay alguno.
- Utilizar funciones en un programa favorece a la reutilización, organización y legibilidad del código.
- Las funciones que declaran su tipo de retorno void pueden implementarse sin necesidad de utilizar la sentencia return.
- Las funciones siempre utilizan parámetros de entrada, ya que su principal tarea es procesar información provista por quien las invoca.
-
-

19

Repaso con preguntas de Teoría

6. Seleccione la afirmación que es falsa:

- El prototipo de una función le indica, formalmente, a la función que la invoca el tipo de dato que será devuelto si es que hay alguno.
- Utilizar funciones en un programa favorece a la reutilización, organización y legibilidad del código.
- Las funciones que declaran su tipo de retorno void pueden implementarse sin necesidad de utilizar la sentencia return.
- *Las funciones siempre utilizan parámetros de entrada, ya que su principal tarea es procesar información provista por quien las invoca.*
-
-

20

Repaso con preguntas de Teoría

1) Enumere las características que debe tener una función para ser considerada recursiva.

2) Describa brevemente por qué existen diferentes algoritmos de búsqueda.

.

.

21

Algoritmos y Estructuras de Datos – AEDD – Asistencia 10/08

**Contraseña del estudiante
mi6kho**

22

Estructuras de datos

- **Simples o básicos:** caracteres, reales, flotantes.
- **Estructurados:** colección de valores relacionados. Se caracterizan por el tipo de dato de sus elementos, la forma de almacenamiento y la forma de acceso.
 - **Estructuras estáticas:** Su tamaño en memoria se mantiene inalterable durante la ejecución del programa, y ocupan posiciones fijas.
ARREGLOS - CADENAS - ESTRUCTURAS
 - **Estructuras dinámicas:** Su tamaño varía durante el programa y no ocupan posiciones fijas.
LISTAS - PILAS - COLAS - ARBOLES - GRAFOS

23

Estructuras de datos - Clasificaciones

- Según donde se almacenan
 - Internas** (en memoria principal)
 - Externas** (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas** (todas del mismo tipo)
 - No homogéneas** (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes** (básicas)
 - Abstractas** (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas** (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas** (su tamaño varía durante el módulo y sus posiciones también)

24

Arreglos Multidimensionales

- Son arreglos que permiten varios índices.
- A los arreglos bidimensionales se los llama comúnmente matrices, tablas o arreglos de arreglos.
- A los de 3 dimensiones, se los denomina cubos
- Y al resto, en general, se los denomina multidimensionales.

Declaración de Arreglos Bidimensionales:

Formato de declaración:

tipo_dato nombre-del-arreglo [nroDeFilas][nroDeColumnas];

- Ejemplo: Definir un arreglo bidimensional de 5 x 3 enteros:

int Matriz [5][3];

25

Arreglos Multidimensionales

- Declaración utilizando tipo estructurado:
- Formato de declaración:

typedef tipo_dato nombre-del-arreglo [dim1][dim2];

- Ejemplo: Definir un arreglo bidimensional de 5 x 3 enteros:

typedef int tMatriz [5][3];

- Creación de variables:

Formato de declaración:

tipo_dato nombre-variable;

Ejemplo: tMatriz m1;

26

Arreglos Multidimensionales

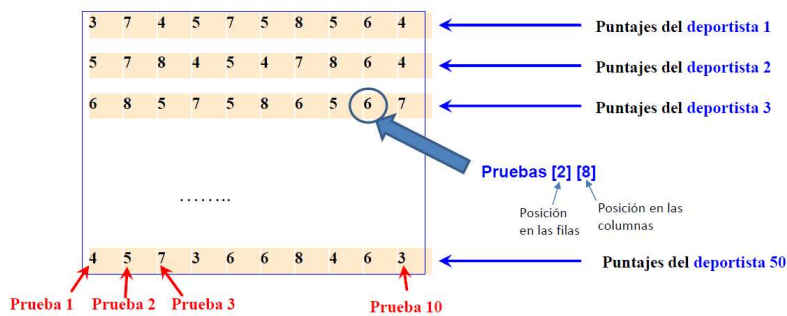
EJEMPLO 1:

- Si se deben almacenar los puntajes de un deportista en 10 pruebas:

`int Pruebas [10];` ← Arreglo

- Si se deben almacenar los puntajes de 50 deportistas en 10 pruebas:

`int Pruebas [50] [10];` ← Matriz

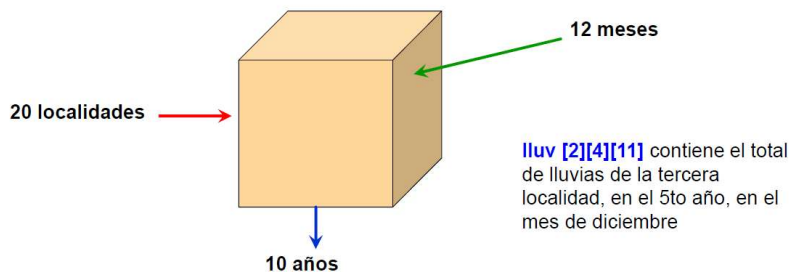


27

Arreglos Multidimensionales

- Las dimensiones pueden ser múltiples.
- Si se desea almacenar los totales mensuales de lluvias mensuales en 20 localidades, durante cada uno de los meses de una década:

`float lluv [20] [10] [12];`



C++ proporciona la posibilidad de almacenar varias dimensiones, aunque raramente los datos del mundo real requieren más de dos o tres dimensiones.

28

Arreglos Multidimensionales

- Supongamos que queremos modelar la temperatura registrada en una cadena de supermercados que tiene sensores en distintas heladeras, ubicadas en varias sucursales y en distintos días del año. Podemos definir un arreglo tridimensional así:

double temperatura[5][10][7];

- **Interpretación:** 5 → cantidad de sucursales. 10 → cantidad de heladeras por sucursal. 7 → cantidad de días registrados.
- Agregamos una 4ta dimensión. Ahora queremos que cada registro no sea sólo una temperatura promedio diaria, sino que tengamos valores cada hora del día (24 horas).

double temperatura[5][10][7][24];

- Agregamos una 5ta dimensión. Ahora sumemos una nueva variable que modele el tipo de producto almacenado en cada heladera. Por ejemplo: 0 = Lácteos, 1 = Carnes, 2 = Congelados, 3 = Bebidas. Entonces:

double temperatura[5][10][7][24][4];

29

Arreglos Multidimensionales

- Como lo interpretamos:
- ***temperatura[sucursal][heladera][día][hora][tipo_producto]*** representa la temperatura registrada en una hora específica, para un tipo de producto determinado, en una heladera, en una sucursal, en un día.

30

Arreglos Multidimensionales - Inicialización

- Por medio de asignaciones:

```
m1[0][0] = 0; m1[0][1] = 0; m1[0][2] = 0;
m1[1][0] = 0; m1[1][1] = 0; m1[1][2] = 0;
```

- Enumerando sus elementos en orden de sus dimensiones:

```
tMatriz m1 = { {0,0,0}, {0,0,0} };
int m1[2][3] = { {0,0,0}, {0,0,0} };
int m1[ ][3] = { {0,0,0}, {0,0,0} };
```

Las llaves separan
filas individuales

En este caso se omite la
longitud de la primera
componente (sólo de la
primera, y no de todas).

31

Matrices: Primer ejercicio

- Determinar si una matriz es nula... todos sus elementos son igual a cero

32

Matrices: Primer ejercicio

- Determinar si una matriz es nula... todos sus elementos son igual a cero

```
bool Es_nula(int mat[][TF], int tl) {
    int i=0, j; bool b=true;
    while((i<tl) && b) {
        j=0;
        while ((j<tl) && b) {
            if (mat[i][j]!=0) b=false;
            j++;
        }
        i++;
    }
    return b;
}
```

33

Algoritmos y Estructuras de Datos – AEDD – Agenda 10/08

- ✓ Problema de Omega de 2do Parcial.
- ✓ Ejercicios de Teoría de 2do Parcial.
- ✓ Revisión de dudas.
- ✓ Arreglos Multidimensionales.

¡¡Objetivo cumplido!!!

34

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



35

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Muy buen inicio del segundo cuatrimestre!



36